

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div> <div>Commander.js</div> <div>Build Status</div> </div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                           | <div> <div> <div>Install Size</div> <div>The complete solution for <a href="#">node.js</a> command-line interfaces, inspired by Ruby's <a href="#">commander</a>.</div> <div>Read this in other languages: English   <a href="#">简体中文</a></div> </div> <ul style="list-style-type: none"> <li>Commander.js <ul style="list-style-type: none"> <li>Installation</li> <li>Declaring <i>program</i> variable</li> <li>Options</li> </ul> </li> <li>Common option types, boolean and value</li> <li>Default option value</li> </ul> </div> | <ul style="list-style-type: none"> <li>Other option types, negatable boolean and <a href="#">flag value</a></li> <li>Custom option processing</li> <li>Required option</li> <li>Version option <ul style="list-style-type: none"> <li>Commands</li> </ul> </li> <li>Specify the argument syntax</li> <li>Action handler (sub)commands</li> <li>Stand-alone executable (sub)commands</li> <li>Automated help</li> <li>Custom help</li> <li>.usage and .name</li> </ul>                                                                                                                                    |
| <div>1</div> <div>Options</div> <div>Options are defined with the .option() method, also serving as documentation for the options. Each option can have a short flag</div> <pre> const { Command } = require('commander'); const program = new Command(); program.version('0.0.1'); </pre> <div>testing, it is better to create a local Command object to use.</div>                                                                                                                                                                                           | <div>7</div> <div>Declaring <i>program</i> variable</div> <div>Commander exports a global object which is convenient for quick programs. This is used in the examples in this README for brevity.</div> <pre> const { program } = require('commander'); program.version('0.0.1'); </pre> <div>For larger programs which may use commander in multiple ways, including unit</div>                                                          | <div>9</div> <div>Installation</div> <div>npm install commander</div> <ul style="list-style-type: none"> <li>Debugging stand-alone executable subcommands</li> <li>Override exit handling</li> <li>Examples</li> <li>License</li> <li>Support</li> <li>Commander for enterprise</li> </ul>                                                                                                                                                                                                                                             | <div>5</div> <ul style="list-style-type: none"> <li>.help(cb)</li> <li>.outputHelp(cb)</li> <li>.helpInformation()</li> <li>.addHelpCommand()</li> <li>Custom event listeners</li> <li>Bits and pieces</li> <li>.parse() and .parseAsync()</li> <li>Avoiding option name clashes</li> <li>TypeScript</li> <li>createCommand()</li> <li>Node options such as --harmony</li> </ul>                                                                                                                                                                                                                         |
| <div>6</div> <div>(single character) and a long name, separated by a comma or space or vertical bar (' ').</div> <div>The options can be accessed as properties on the Command object. Multi-word options such as “–template-engine” are camel-cased, becoming program.templateEngine etc. See also optional new behaviour to <a href="#">avoid name clashes</a>.</div> <div>Multiple short flags may optionally be combined in a single argument following the dash: boolean flags, the last flag may take a value, and the value. For example -a -b -p</div> | <div>10</div> <div>80 may be written as -ab -p80 or even -abp80.</div> <div>You can use -- to indicate the end of the options, and any remaining arguments will be used without being interpreted. This is particularly useful for passing options through to another command, like: do -- git -- version.</div> <div>Options on the command line are not positional, and can be specified before or after other command arguments.</div> | <div>11</div> <div>Common option types, boolean and value</div> <div>The two most used option types are a boolean flag, and an option which takes a value (declared using angle brackets). Both are undefined unless specified on command line.</div> <pre> const { program } = require('commander'); </pre>                                                                                                                                                                                                                           | <div>12</div> <pre> program   .option('-d, --debug',     'output extra debugging')   .option('-s, --small', 'small pizza size')   .option('-p, --pizza-type &lt;type&gt;',     'flavour of pizza');  program.parse(process.argv);  if (program.debug)   console.log(program.opts()); </pre>                                                                                                                                                                                                                                                                                                              |
| <div>16</div> <pre> program.parse(process.argv);  console.log(`cheese: \${program.cheese}`); \$ pizza-options cheese: blue \$ pizza-options --cheese stilton cheese: stilton </pre>                                                                                                                                                                                                                                                                                                                                                                            | <div>15</div> <div>Default option value</div> <div>You can specify a default value for an option which takes a value.</div> <pre> const { program } = require('commander'); </pre> <div>by the program options in the program.args array.</div>                                                                                                                                                                                           | <div>14</div> <pre> \$ pizza-options -ds -p vegetarian { debug: true, small: true,   pizzaType: 'vegetarian' } pizza details: - small pizza size - vegetarian \$ pizza-options --pizza-type=cheese pizza details: - cheese program.parse(arguments) processes the arguments, leaving any args not consumed </pre>                                                                                                                                                                                                                      | <div>13</div> <pre> console.log('pizza details:'); if (program.small) console.log('-small pizza size'); if (program.pizzaType) console.log(`- \${program.pizzaType}`); \$ { debug: true, small: undefined,   pizzaOptions: { debug: true, small: undefined,     pizzaType: undefined }   } \$ { debug: true, small: undefined,   pizzaOptions: { debug: true, small: undefined,     pizzaType: undefined }   } \$ { debug: true, small: undefined,   pizzaOptions: { debug: true, small: undefined,     pizzaType: undefined }   } error: option '-p, --pizza-type' argument missing &lt;type&gt; </pre> |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div data-bbox="62 49 427 103" data-label="Section-Header"> <h3>Other option types, negatable boolean and flag value</h3> </div> <div data-bbox="62 127 495 336" data-label="Text"> <p>You can specify a boolean option long name with a leading no- to set the option value to false when used. Defined alone this also makes the option true by default.</p> <p>If you define --foo first, adding --no-foo does not change the default value from what it would otherwise be. You can specify a default boolean value for a boolean flag and it can be overridden on command line.</p> </div> <div data-bbox="271 367 288 383" data-label="Text"> <p>17</p> </div> | <div data-bbox="622 49 1023 328" data-label="Text"> <pre>const { program } =   require('commander');  program   .option('--no-sauce', 'Remove     sauce')   .option('--cheese &lt;flavour&gt;',     'cheese flavour',     'mozzarella')   .option('--no-cheese', 'plain with     no cheese')   .parse(process.argv);</pre> </div> <div data-bbox="831 367 848 383" data-label="Text"> <p>18</p> </div>                                                                                                                                                                                               | <div data-bbox="1182 73 1594 344" data-label="Text"> <pre>const sauceStr = program.sauce ?   'sauce' : 'no sauce'; const cheeseStr = (program.cheese ===   false) ? 'no cheese' : `\${   {program.cheese} cheese`; console.log(`You ordered a pizza with   \${sauceStr} and \${cheeseStr}   `);  \$ pizza-options You ordered a pizza with sauce and   mozzarella cheese</pre> </div> <div data-bbox="1391 367 1408 383" data-label="Text"> <p>19</p> </div>                                                                               | <div data-bbox="1742 49 2168 320" data-label="Text"> <pre>\$ pizza-options --sauce error: unknown option '--sauce' \$ pizza-options --cheese=blue You ordered a pizza with sauce and   blue cheese \$ pizza-options --no-sauce --no-cheese You ordered a pizza with no sauce and   no cheese  You can specify an option which functions as a flag but may also take a value (declared using square brackets).</pre> </div> <div data-bbox="1951 367 1968 383" data-label="Text"> <p>20</p> </div> |
| <div data-bbox="107 451 499 746" data-label="Text"> <pre>} return parseInt(value); optional radix // parseInt takes a string and an dummyPrevious) { function myParseInt(value, require('commander'); const { program } = function. You can optionally specify the default/ starting value for the option after the</pre> </div> <div data-bbox="271 414 288 430" data-label="Text"> <p>24</p> </div>                                                                                                                                                                                                                                                                | <div data-bbox="629 488 1057 743" data-label="Text"> <p>You may specify a function to do custom processing of option values. The callback function receives two parameters, the user specified value and the previous value for the option. It returns the new value for the option. This allows you to coerce the option value to the desired type, or accumulate values, or do entirely custom processing.</p> </div> <div data-bbox="741 719 1057 743" data-label="Section-Header"> <h3>Custom option processing</h3> </div> <div data-bbox="831 414 848 430" data-label="Text"> <p>23</p> </div> | <div data-bbox="1227 496 1617 743" data-label="Text"> <pre>else if (program.cheese === true)   console.log('add cheese'); else console.log('add cheese type \$   {program.cheese}'); \$ pizza-options no cheese \$ pizza-options --cheese add cheese \$ pizza-options --cheese mozzarella add cheese type mozzarella</pre> </div> <div data-bbox="1391 414 1408 430" data-label="Text"> <p>22</p> </div>                                                                                                                                   | <div data-bbox="1765 488 2177 743" data-label="Text"> <pre>const { program } =   require('commander'); program   .option('-c, --cheese [type]', 'Add     cheese with optional type'); program.parse(process.argv); if (program.cheese === undefined)   console.log('no cheese');</pre> </div> <div data-bbox="1951 414 1968 430" data-label="Text"> <p>21</p> </div>                                                                                                                              |
| <div data-bbox="62 871 477 1078" data-label="Text"> <pre>function   increaseVerbosity(dummyValue,     previous) {   return previous + 1; }  function collect(value, previous) {   return previous.concat([value]); }</pre> </div> <div data-bbox="271 1165 288 1181" data-label="Text"> <p>25</p> </div>                                                                                                                                                                                                                                                                                                                                                             | <div data-bbox="622 847 1034 1102" data-label="Text"> <pre>function commaSeparatedList(value,   dummyPrevious) {   return value.split(','); }  program   .option('-f, --float &lt;number&gt;',     'float argument', parseFloat)   .option('-i, --integer &lt;number&gt;',     'integer argument',     myParseInt)</pre> </div> <div data-bbox="831 1165 848 1181" data-label="Text"> <p>26</p> </div>                                                                                                                                                                                               | <div data-bbox="1182 847 1594 1126" data-label="Text"> <pre>.option('-v, --verbose', 'verbosity   that can be increased',   increaseVerbosity, 0) .option('-c, --collect &lt;value&gt;',   'repeatable value', collect,   []) .option('-l, --list &lt;items&gt;',   'comma separated list',   commaSeparatedList) ;  program.parse(process.argv);</pre> </div> <div data-bbox="1391 1165 1408 1181" data-label="Text"> <p>27</p> </div>                                                                                                    | <div data-bbox="1742 871 2119 1078" data-label="Text"> <pre>if (program.float !== undefined)   console.log(`float: \$     {program.float}`); if (program.integer !== undefined)   console.log(`integer: \$     {program.integer}`); if (program.verbose &gt; 0)   console.log(`verbosity: \$     {program.verbose}`);</pre> </div> <div data-bbox="1951 1165 1968 1181" data-label="Text"> <p>28</p> </div>                                                                                       |
| <div data-bbox="62 1241 499 1544" data-label="Text"> <p>The optional version method adds handling for displaying the command version. The default option flags are -V and --version, and when present the command prints the version number and exits.</p> <p>\$ pizza error: required option '-c, --cheese &lt;type&gt;' not specified</p> </div> <div data-bbox="320 1430 499 1453" data-label="Section-Header"> <h3>Version option</h3> </div> <div data-bbox="271 1212 288 1228" data-label="Text"> <p>32</p> </div>                                                                                                                                             | <div data-bbox="667 1270 1057 1541" data-label="Text"> <pre>const { program } =   require('commander'); description, and optional default value or custom processing. program   .requiredOption('-c, --cheese     &lt;type&gt;', 'pizza must have     cheese'); program.parse(process.argv);</pre> </div> <div data-bbox="831 1212 848 1228" data-label="Text"> <p>31</p> </div>                                                                                                                                                                                                                     | <div data-bbox="1193 1262 1617 1541" data-label="Text"> <p>You may specify a required (mandatory) option using .requiredOption(). The option must have a value after parsing, usually specified on the command line, or perhaps from a default value (say from environment). The method is otherwise the same as .option in format, taking flags and</p> </div> <div data-bbox="1424 1453 1617 1477" data-label="Section-Header"> <h3>Required option</h3> </div> <div data-bbox="1391 1212 1408 1228" data-label="Text"> <p>30</p> </div> | <div data-bbox="1765 1246 2177 1541" data-label="Text"> <pre>if (program.collect.length &gt; 0)   console.log(program.collect); if (program.list !== undefined)   console.log(program.list); \$ custom -f 1e2 float: 100 \$ custom --integer 2 integer: 2 \$ custom -v -v -v verbosity: 3 \$ custom -c a -c b -c c [ 'a', 'b', 'c' ]</pre> </div> <div data-bbox="1951 1212 1968 1228" data-label="Text"> <p>29</p> </div>                                                                        |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div data-bbox="62 49 297 95" data-label="Text"> <pre>\$ ./examples/pizza -V 0.0.1</pre> </div> <div data-bbox="62 116 495 256" data-label="Text"> <p>You may change the flags and description by passing additional parameters to the version method, using the same syntax for flags as the option method. The version flags can be named anything, but a long name is required.</p> </div> <div data-bbox="62 277 445 344" data-label="Text"> <pre>program.version('0.0.1', '-v, --   vers', 'output the current   version');</pre> </div> <div data-bbox="271 367 286 383" data-label="Text"> <p>33</p> </div> | <div data-bbox="775 49 907 73" data-label="Section-Header"> <h3>Commands</h3> </div> <div data-bbox="622 95 1055 256" data-label="Text"> <p>You can specify (sub)commands using .command() or .addCommand(). There are two ways these can be implemented: using an action handler attached to the command, or as a stand-alone executable file (described in more detail later). The subcommands may be nested (<a href="#">example</a>).</p> <p>In the first parameter to .command() you specify the command name and any command arguments. The arguments may be</p> </div> <div data-bbox="831 367 846 383" data-label="Text"> <p>34</p> </div> | <div data-bbox="1182 49 1563 167" data-label="Text"> <p>&lt;required&gt; or [optional], and the last argument may also be variadic....</p> <p>You can use .addCommand() to add an already configured subcommand to the program.</p> <p>For example:</p> </div> <div data-bbox="1182 212 1572 351" data-label="Text"> <pre>// Command implemented using action   handler (description is   supplied separately to   '.command') // Returns new command for   configuring.</pre> </div> <div data-bbox="1391 367 1406 383" data-label="Text"> <p>35</p> </div> | <div data-bbox="1742 49 2145 280" data-label="Text"> <pre>program   .command('clone &lt;source&gt;     [destination]')   .description('clone a repository     into a newly created     directory')   .action((source, destination) =&gt; {     console.log('clone command       called');   });</pre> </div> <div data-bbox="1951 367 1966 383" data-label="Text"> <p>36</p> </div>                                                                                                       |
| <div data-bbox="271 414 286 430" data-label="Text"> <p>40</p> </div> <div data-bbox="152 469 499 743" data-label="Text"> <pre>program.parse(process.argv);   });   envValue = env;   cmdValue = cmd;   .action(function (cmd, env) {     .arguments('&lt;cmd&gt; [env]'     )     .version('0.1.0')     program     const { program } =       require('commander');</pre> </div>                                                                                                                                                                                                                                   | <div data-bbox="831 414 846 430" data-label="Text"> <p>39</p> </div> <div data-bbox="622 462 1059 743" data-label="Text"> <p>will run the subcommand if no other subcommand is specified (<a href="#">example</a>).</p> <p><b>Specify the argument syntax</b></p> <p>You use .arguments to specify the arguments for the top-level command, and for subcommands they are usually included in the .command call. Angled brackets (e.g. &lt;required&gt;) indicate required input. Square brackets (e.g. [optional]) indicate optional input.</p> </div>                                                                                             | <div data-bbox="1391 414 1406 430" data-label="Text"> <p>38</p> </div> <div data-bbox="1173 469 1619 721" data-label="Text"> <pre>program   .addCommand(buidl.makeBuidlCommand());   // Command prepared separately.   // Returns `this` for adding more   commands.</pre> <p>Configuration options can be passed with the call to .command() and .addCommand(). Specifying true for opts.hidden will remove the command from the generated help output. Specifying true for opts.isDefault</p> </div>                                                       | <div data-bbox="1951 414 1966 430" data-label="Text"> <p>37</p> </div> <div data-bbox="1778 469 2179 743" data-label="Text"> <pre>// Command implemented using stand-   alone executable file   (description is second   parameter to '.command')   // Returns `this` for adding more   commands.   program     .command('start &lt;service&gt;', 'start       named service')     .command('stop [service]', 'stop       named service, or all if no       name supplied');</pre> </div> |
| <div data-bbox="271 1165 286 1181" data-label="Text"> <p>41</p> </div> <div data-bbox="62 871 490 1031" data-label="Text"> <pre>if (typeof cmdValue === 'undefined') {   console.error('no command given!');   process.exit(1); } console.log('command:', cmdValue); console.log('environment:', envValue      "no environment given");</pre> </div> <div data-bbox="62 1053 495 1145" data-label="Text"> <p>The last argument of a command can be variadic, and only the last argument. To make an argument variadic you append ... to the argument name. For example:</p> </div>                                 | <div data-bbox="831 1165 846 1181" data-label="Text"> <p>42</p> </div> <div data-bbox="622 849 1037 1123" data-label="Text"> <pre>const { program } =   require('commander');  program   .version('0.1.0')   .command('rmdir &lt;dir&gt;     [otherDirs...]' )   .action(function (dir, otherDirs) {     console.log('rmdir %s', dir);     if (otherDirs) {       otherDirs.forEach(function         (oDir) {</pre> </div>                                                                                                                                                                                                                         | <div data-bbox="1391 1165 1406 1181" data-label="Text"> <p>43</p> </div> <div data-bbox="1182 849 1606 984" data-label="Text"> <pre>console.log('rmdir %s', oDir);   });   }   });  program.parse(process.argv);</pre> <p>The variadic argument is passed to the action handler as an array.</p> </div>                                                                                                                                                                                                                                                      | <div data-bbox="1742 849 2128 873" data-label="Section-Header"> <h3>Action handler (sub)commands</h3> </div> <div data-bbox="1742 895 2157 1082" data-label="Text"> <p>You can add options to a command that uses an action handler. The action handler gets passed a parameter for each argument you declared, and one additional argument which is the command object itself. This command argument has the values for the command-specific options added as properties.</p> </div>     |
| <div data-bbox="271 1212 286 1228" data-label="Text"> <p>48</p> </div> <div data-bbox="62 1278 499 1541" data-label="Text"> <p><b>Stand-alone executable (sub)commands</b></p> <p>When .command() is invoked with a description argument, this tells Commander that you're going to use stand-alone executables for subcommands. Commander will search the executables in the directory of the entry script (like ./examples/pm) with the name program-subcommand, like pm-install, pm-search. You can specify a custom name</p> </div>                                                                            | <div data-bbox="831 1212 846 1228" data-label="Text"> <p>47</p> </div> <div data-bbox="600 1278 1059 1541" data-label="Text"> <pre>.action(run);   await program.parseAsync(process.argv);   }    A command's options on the command line are validated when the command is used. Any unknown options will be reported as an error.</pre> </div>                                                                                                                                                                                                                                                                                                   | <div data-bbox="1391 1212 1406 1228" data-label="Text"> <p>46</p> </div> <div data-bbox="1191 1251 1619 1519" data-label="Text"> <pre>program.parse(process.argv)    You may supply an async action handler, in which case you call .parseAsync rather than .parse.    async function run() { /* code goes     here */ }   async function main() {     program     .command('run')     }</pre> </div>                                                                                                                                                        | <div data-bbox="1951 1212 1966 1228" data-label="Text"> <p>45</p> </div> <div data-bbox="1778 1267 2179 1541" data-label="Text"> <pre>const { program } =   require('commander');    program     .command('rm &lt;dir&gt;' )     .option('-r, --recursive', 'Remove       recursively')     .action(function (dir, cmdObj) {       console.log('remove ' + dir +         cmdObj.recursive ?         'recursively' : '');     })     }</pre> </div>                                        |

|                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>with the executableFile configuration option.</p> <p>You handle the options for an executable (sub)command in the executable, and don't declare them at the top-level.</p> <pre>// file: ./examples/pm const { program } =   require('commander');</pre> <pre>program   .version('0.1.0')</pre>             | <pre>.command('install [name]', 'install   one or more packages') .command('search [query]', 'search   with optional query') .command('update',   'update installed packages',   {executableFile:     'myUpdateSubCommand'}) .command('list', 'list packages   installed', {isDefault: true}) .parse(process.argv);</pre> | <p>If the program is designed to be installed globally, make sure the executables have proper modes, like 755.</p> <p><b>Automated help</b></p> <p>The help information is auto-generated based on the information commander already knows about your program. The default help option is -h, --help. (example)</p> <pre>\$ node ./examples/pizza --help Usage: pizza [options]</pre> | <p>An application for pizzas ordering</p> <p>Options:</p> <pre>-V, --version      output the                     version number -p, --peppers      Add peppers -c, --cheese &lt;type&gt; Add the                     specified type of cheese                     (default: "marble")</pre>                                                         |
| <pre>Usage: custom-help [options]  Options:   -f, --foo      enable some foo   -h, --help     display help for command Example call: \$ custom-help --help</pre>                                                                                                                                               | <pre>program   .option('-f, --foo', 'enable some     foo'); // must be before .parse() program.on('--help', () =&gt; {   console.log('');   console.log('Example call:');   console.log('\$ custom-help --     help'); }); Yields the following help output:</pre>                                                        | <pre>shell help shell --help shell spawn shell help spawn You can display extra information by listening for "--help", (example)</pre> <p><b>Custom help</b></p>                                                                                                                                                                                                                      | <pre>-C, --no-cheese     You do not want     any cheese -h, --help     display help     for command</pre> <p>A help command is added by default if you command has subcommands. It can be used alone, or with a subcommand name to show further help for the subcommand. These are effectively the same if the shell program has implicit help:</p> |
| <p><b>.usage and .name</b></p> <p>These allow you to customise the usage description in the first line of the help. The name is otherwise deduced from the (full) program arguments. Given:</p> <pre>program   .name("my-command")   .usage("[global options] command")</pre> <p>The help will start with:</p> | <pre>Usage: my-command [global options] command</pre> <p><b>.help(cb)</b></p> <p>Output help information and exit immediately. Optional callback cb allows post-processing of help text before it is displayed.</p>                                                                                                       | <p><b>.outputHelp(cb)</b></p> <p>Output help information without exiting. Optional callback cb allows post-processing of help text before it is displayed.</p> <p><b>.helpInformation()</b></p> <p>Get the command help information as a string for processing or displaying yourself. (The text does not include the custom help from --help listeners.)</p>                         | <p><b>.helpOption(flags, description)</b></p> <p>Override the default help flags and description.</p> <pre>program   .helpOption('-e, --HELP',     'read more information');</pre>                                                                                                                                                                  |
| <p><b>Bits and pieces</b></p> <p><b>.parse() and .parseAsync()</b></p> <p>The first argument to .parse is the array of strings to parse. You may omit the parameter to implicitly use process.argv.</p>                                                                                                        | <pre>console.error(`error: unknown   command '\${operands[0]}') const availableCommands =   program.commands.map(cmd =&gt;     cmd.name()); mySuggestsBestMatch(operands[0],   availableCommands); process.exitCode = 1; }});</pre>                                                                                       | <p><b>Custom event listeners</b></p> <p>You can execute custom actions by listening to command and option events.</p> <pre>program.on('option:verbose', function   () {     process.env.VERBOSE = this.verbose;   }); program.on('command:*', function   (operands) {     // ...   });</pre>                                                                                          | <p><b>.addHelpCommand()</b></p> <p>You can explicitly turn on or off the implicit help command with .addHelpCommand() and .addHelpCommand(false).</p> <p>You can both turn on and customise the help command by supplying the name and description:</p> <pre>program.addHelpCommand('assist   [command]', 'show   assistance');</pre>               |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>If the arguments follow different conventions than node you can pass a from option in the second parameter:</p> <ul style="list-style-type: none"> <li>• `node`: default, argv[0] is the application and argv[1] is the script being run, with user parameters after that</li> <li>• `electron`: argv[1] varies depending on whether the electron application is packaged</li> <li>• `user`: all of the arguments from the user</li> </ul> <p>For example:</p> | <pre>program.parse(process.argv); //     Explicit, node conventions program.parse(); // Implicit, and     auto-detect electron program.parse(['-f', 'filename'], {     from: 'user' });</pre> <p><b>Avoiding option name clashes</b></p> <p>The original and default behaviour is that the option values are stored as properties on the program, and the action handler is passed</p>                                | <p>a command object with the options values stored as properties. This is very convenient to code, but the downside is possible clashes with existing properties of Command.</p> <p>There are two new routines to change the behaviour, and the default behaviour may change in the future:</p> <ul style="list-style-type: none"> <li>• storeOptionsAsProperties: whether to store option values as properties on command object, or store separately (specify false) and access using .opts()</li> </ul> | <ul style="list-style-type: none"> <li>• passCommandToAction: whether to pass command to action handler, or just the options (specify false) (<a href="#">example</a>)</li> </ul> <pre>program     .storeOptionsAsProperties(false)     .passCommandToAction(false);  program     .name('my-program-name')     .option('-n,--name &lt;name&gt;');</pre>                                                                       |
| <p>gets used internally when creating subcommands using .command(), and you may override it to customise the new subcommand (examples using <a href="#">subclass</a> and <a href="#">function</a>).</p>                                                                                                                                                                                                                                                           | <p>This factory function creates a new command. It is exported and may be used instead of using new, like:</p> <pre>const { createCommand } =     require('commander'); const program = createCommand();  createCommand is also a method of the Command object, and creates a new command rather than a subcommand. This </pre>                                                                                       | <p>The Commander package includes its TypeScript Definition file. If you use ts-node and stand-alone executable subcommands written as .ts files, you need to call your program through node to get the subcommands called correctly, e.g.</p> <pre>node -r ts-node/register pm.ts</pre>                                                                                                                                                                                                                   | <pre>program     .command('show')     .option('-a,--action &lt;action&gt;')     .action((options) =&gt; {         console.log(options.action);     }); program.parse(process.argv); const programOptions = program.opts(); console.log(programOptions.name);</pre>                                                                                                                                                            |
| <p><b>Node options such as --harmony</b></p> <p>You can enable --harmony option in two ways:</p> <ul style="list-style-type: none"> <li>• Use #! /usr/bin/env node --harmony in the subcommands scripts. (Note Windows does not support this pattern.)</li> <li>• Use the --harmony option when call the command, like</li> </ul> <pre>node --harmony examples/pm publish.</pre> <p>The --harmony option will be preserved when spawning subcommand process.</p>  | <p><b>Debugging stand-alone executable subcommands</b></p> <p>An executable subcommand is launched as a separate child process.</p> <p>If you are using the node inspector for <a href="#">debugging</a> executable subcommands using node --inspect et al, the inspector port is incremented by 1 for the spawned subcommand.</p> <p>If you are using VSCode to debug executable subcommands you need to set the</p> | <p>"autoAttachChildProcesses": true flag in your launch.json configuration.</p> <p><b>Override exit handling</b></p> <p>By default Commander calls process.exit when it detects errors, or after displaying the help or version. You can override this behaviour and optionally supply a callback. The default override throws a CommanderError.</p>                                                                                                                                                       | <p>The override callback is passed a CommanderError with properties exitCode number, code string, and message. The default override behaviour is to throw the error, except for async handling of executable subcommand completion which carries on. The normal display of error messages or version or help is not affected by the override which is called after the display.</p> <pre>program.exitOverride();  try {</pre> |
| <pre>program     .command('exec &lt;cmd&gt;')     .alias('ex')     .description('execute the given         remote cmd')     .option('-e,--exec-mode &lt;mode&gt;',         'which exec mode to use')     .action(function(cmd, options) {         console.log('exec "%s" using %s             mode', cmd,             options.execMode);     }).on('--help', function() {     });</pre>                                                                           | <pre>.description('run setup commands for     all envs') .option('-s,--setup-mode [mode]',     'which setup mode to use') .action(function(env, options) {     const mode = options.setup_mode         : "normal"         : env    'all';     console.log('setup for %s env(s)         with %s mode', env, mode);     });</pre>                                                                                       | <pre>program     .command('setup [env]',         'change the working         directory')     .option('-C,--chdir &lt;path&gt;',         'config path, defaults to . /         deploy.conf')     .option('-T,--no-tests', 'ignore         test hook');</pre>                                                                                                                                                                                                                                                | <p><b>Examples</b></p> <pre>program.parse(process.argv); catch (err) {     // custom processing... }  const { program } =     require('commander');</pre> <pre>program     .version('0.1.0')     .command('examples')</pre>                                                                                                                                                                                                   |

|                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| <pre>console.log(''); console.log('Examples:'); console.log(''); console.log('  \$ deploy exec     sequential'); console.log('  \$ deploy exec     async');  });  program.parse(process.argv);  More Demos can be found in the <a href="#">examples</a> directory.</pre> <div>81</div> | <div>License</div> <div><a href="#">MIT</a></div> <div>Support</div> <div>Commander 5.x is fully supported on Long Term Support versions of Node, and is likely to work with Node 6 but not tested. (For versions of Node below Node 6, use Commander 3.x or 2.x.)</div> <div>The main forum for free and community support is the project <a href="#">Issues</a> on GitHub.</div> <div>82</div> | <div>Commander for enterprise</div> <div>Available as part of the Tidelift Subscription</div> <div>The maintainers of Commander and thousands of other packages are working with Tidelift to deliver commercial support and maintenance for the open source dependencies you use to build your applications. Save time, reduce risk, and improve code health, while paying the maintainers of the exact dependencies you use. <a href="#">Learn more.</a></div> <div>83</div> | <div>84</div> |
| <div>88</div>                                                                                                                                                                                                                                                                          | <div>£8</div>                                                                                                                                                                                                                                                                                                                                                                                    | <div>98</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | <div>£8</div> |
| <div>89</div>                                                                                                                                                                                                                                                                          | <div>90</div>                                                                                                                                                                                                                                                                                                                                                                                    | <div>91</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | <div>92</div> |
| <div>96</div>                                                                                                                                                                                                                                                                          | <div>£6</div>                                                                                                                                                                                                                                                                                                                                                                                    | <div>₹6</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | <div>£6</div> |